Customer Churn Prediction Report

HarvardX PH125.9x - Data Science: Capstone

Eman Ahmad

June 13, 2026

Contents

1 Introduction / Overview

Customer churn - when a paying customer stops using a product or cancels their subscription - is one of the most common problems businesses try to predict, because it is usually a lot cheaper to keep an existing customer than to acquire a new one. For this project I picked a business/SaaS style customer churn dataset that contains demographic information, usage behaviour, billing history, support interactions and satisfaction scores for 10,000 customers, along with a binary label that tells us whether each customer churned (`1`) or not (`0`).

The goal of this project is to build a model that, given information about a customer, can predict whether that customer is likely to churn. Being able to flag at-risk customers ahead of time would let a business target them with retention offers, follow-up calls, discounts, etc.

1.1 Dataset

The dataset has 10,000 rows and 32 columns. After dropping the customer ID column (which is just an identifier and has no predictive value), we are left with 31 variables: 1 target variable (churn) and 30 predictors. The predictors fall roughly into a few groups:

- **Demographics**: gender, age, country, city, customer_segment
- **Account info**: tenure_months, signup_channel, contract_type
- **Product usage**: monthly_logins, weekly_active_days, avg_session_time, features_used, usage_growth_rate, last_login_days_ago
- **Billing**: monthly_fee, total_revenue, payment_method, payment_failures, discount_applied, price_increase_last_3m
- **Support / satisfaction**: support_tickets, avg_resolution_time, complaint_type, csat_score, escalations
- **Marketing / engagement**: email_open_rate, marketing_click_rate, nps_score, survey_response, referral_count

The target variable churn is fairly imbalanced - only about 10.2% of the customers in the dataset actually churned. This is realistic (most companies don't lose 50% of their customers!) but it does make the prediction problem harder, and I'll come back to this point in the results and conclusion sections.

1.2 Key steps performed

1. Load and clean the dataset (handle the one column that has missing values, convert categorical columns to factors).
2. Explore the data with summary statistics and plots, to get a feel for which variables seem related to churn.
3. Split the data into a training set (80%) and a test set (20%).
4. Train two different models on the training set: a logistic regression model and a random forest model.
5. Evaluate both models on the test set using accuracy, sensitivity, specificity and the F1 score, and compare them.

2 Methods / Analysis

2.1 Data cleaning

The only column with missing values is complaint_type, which has 2,045 missing entries out of 10,000. Looking at the other columns, this makes sense - a customer who never filed a complaint simply doesn't have a complaint type. Rather than dropping these rows (which would mean losing about 20% of the data), I recoded the missing values as "None", i.e. an extra level meaning "no complaint on file".

```
df <- read_csv("customer_churn_business_dataset.csv")

# the only column with NAs
colSums(is.na(df))

df <- df %>%
  mutate(complaint_type = ifelse(is.na(complaint_type), "None", complaint_type)) %>%
  select(-customer_id)
```

After that, I converted churn and the categorical columns to factors so that caret and the underlying models would treat them correctly.

```
df <- df %>%
  mutate(
    churn = factor(churn, levels = c(0,1), labels = c("No","Yes")),
    gender = factor(gender),
    country = factor(country),
    city = factor(city),
    customer_segment = factor(customer_segment),
    signup_channel = factor(signup_channel),
    contract_type = factor(contract_type),
    payment_method = factor(payment_method),
    discount_applied = factor(discount_applied),
    price_increase_last_3m = factor(price_increase_last_3m),
    complaint_type = factor(complaint_type),
    survey_response = factor(survey_response,
                             levels = c("Unsatisfied","Neutral","Satisfied"))
  )
```

2.2 Exploratory data analysis

2.2.1 Overall churn rate

The first thing I looked at was the overall churn rate, just to know what I'm dealing with.

```
df %>%
  count(churn) %>%
  mutate(pct = n / sum(n)) %>%
  ggplot(aes(churn, pct, fill = churn)) +
  geom_col() +
  geom_text(aes(label = percent(pct)), vjust = -0.3) +
  scale_y_continuous(labels = percent) +
  labs(title = "Overall churn rate", x = "Churn", y = "Percentage of customers") +
  theme_minimal()
```

Only about 10.2% of customers churned. This class imbalance is something I kept in mind for the rest of the analysis - a model that just predicted "No" for every customer would already get about 90% accuracy, so accuracy alone isn't going to be a great way to judge how useful a model actually is.

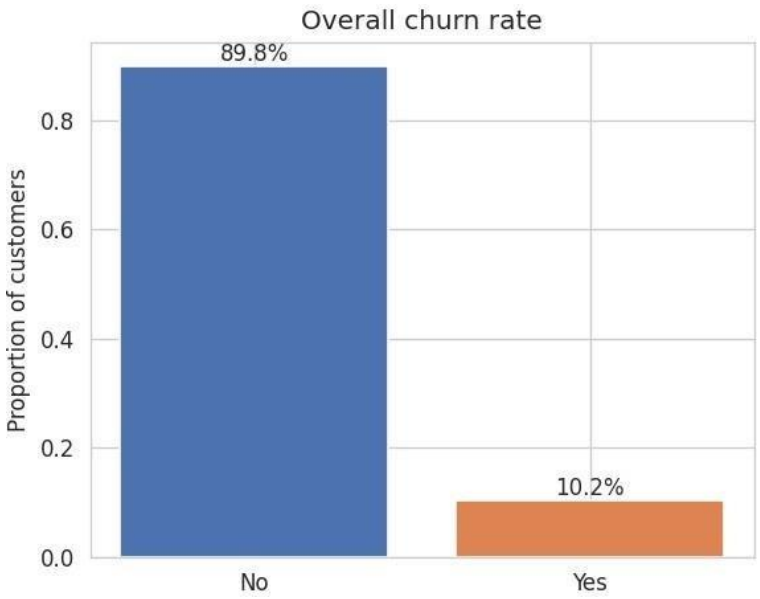

Figure 1: Overall churn rate

2.2.2 Churn by contract type and customer segment

I expected contract type (monthly vs quarterly vs yearly) to be a strong predictor, since customers locked into longer contracts often have a harder time leaving. The actual numbers were a bit surprising:

```
df %>%
  group_by(contract_type)  %>%
  summarise(churn_rate = mean(churn == "Yes")) %>%
  ggplot(aes(contract_type, churn_rate, fill = contract_type)) +
  geom_col() +
  scale_y_continuous(labels = percent) +
  labs(title = "Churn rate by contract type", x = "Contract type", y = "Churn rate") +
  theme_minimal()
```

Monthly, quarterly and yearly contracts all churn at very close to the same rate (around 10%). Customer segment tells a similar story:

SME customers churn slightly more than Enterprise or Individual customers (about 10.9% vs 9.4% and 10.0%), but the difference is small. Neither of these variables on its own looks like it's going to be a strong predictor.

2.2.3 Tenure

Tenure (how many months a customer has been with the company) showed a more noticeable pattern - customers who churned tend to have shorter tenure on average (about 24.2 months vs 30.8 months for customers who stayed).

2.2.4 Satisfaction scores (CSAT and NPS)

CSAT (customer satisfaction score) also looked like a useful signal - customers who churned had a lower average CSAT score (about 3.03 vs 3.54 for customers who stayed).

NPS (net promoter score), on the other hand, was basically identical between the two groups (about 19.5 for churners vs 19.1 for non-churners), which I honestly didn't expect - I would have guessed that customers who are about to leave would also be giving lower NPS scores.

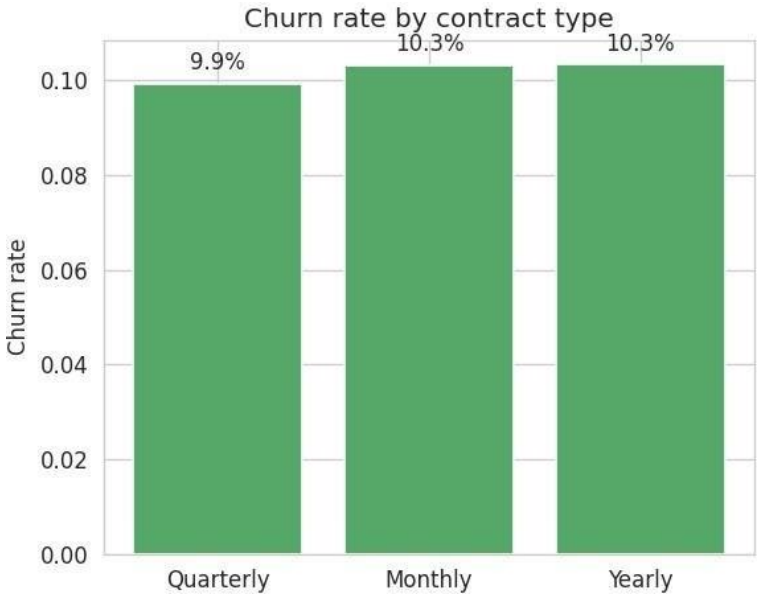

Figure 2: Churn rate by contract type

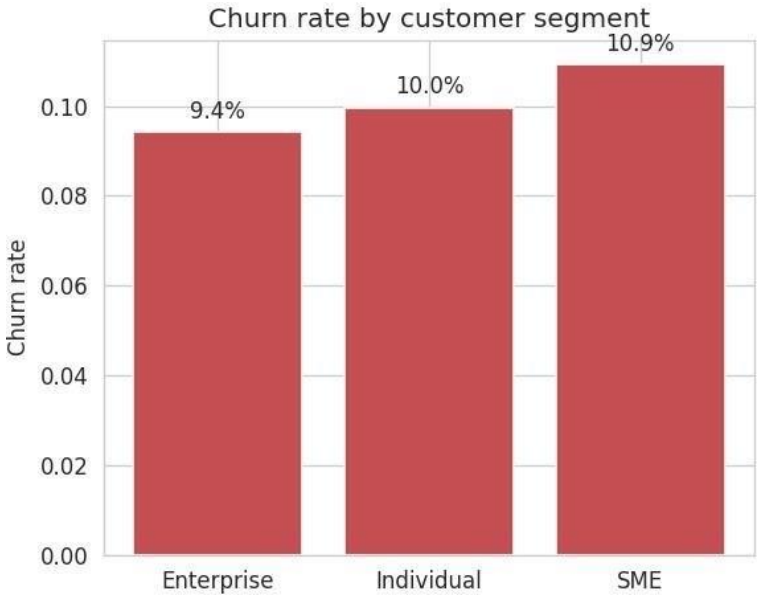

Figure 3: Churn rate by customer segment

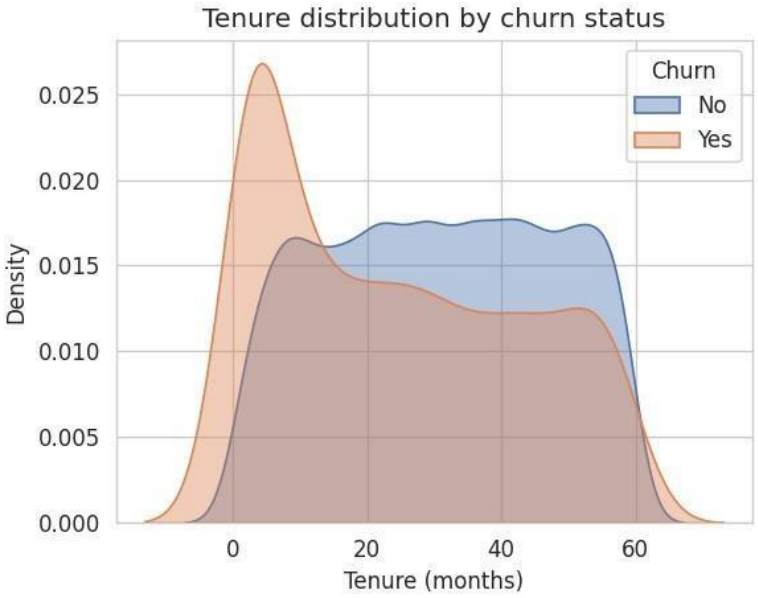

Figure 4: Tenure distribution by churn status

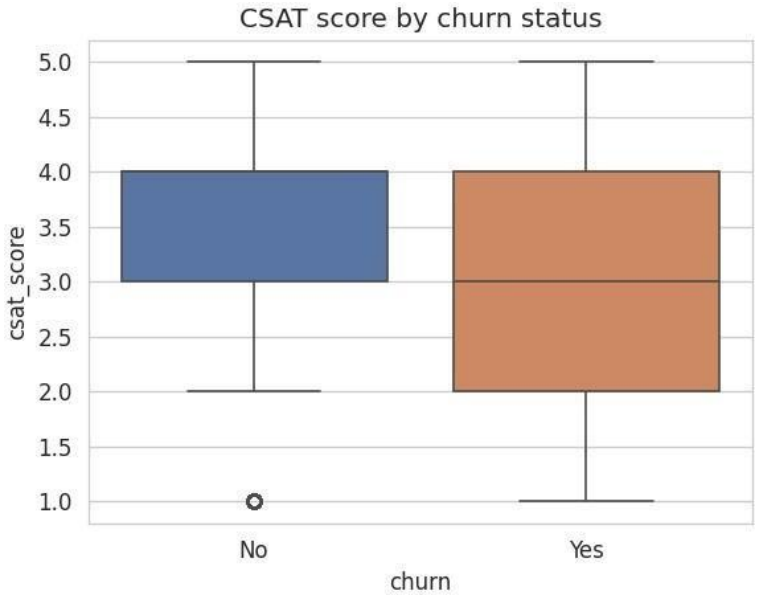

Figure 5: CSAT score by churn status

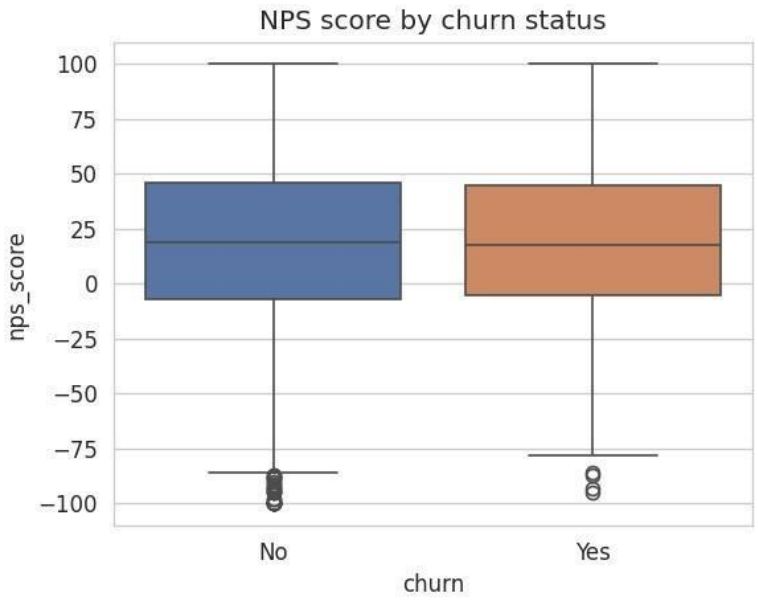

Figure 6: NPS score by churn status

2.2.5 Correlation between numeric variables

To check for any obviously redundant variables, I looked at a correlation matrix of all the numeric predictors.

Nothing stood out as a near-duplicate pair of columns, so I kept all of the numeric variables for modeling.

2.2.6 Support tickets and payment failures

Support tickets showed only a weak relationship with churn. Customers with three support tickets had a slightly higher churn rate than other groups, but overall the differences were relatively small

2.3 Modeling approach

I split the data into a training set (80%, ~8,000 rows) and a test set (20%, ~2,000 rows), making sure the churn rate stayed roughly the same in both sets (using `createDataPartition` with the `churn` column).

```
set.seed(1, sample.kind = "Rounding")
test_index <- createDataPartition(df$churn, times = 1, p = 0.2, list = FALSE)
train_set <- df[-test_index, ]
test_set  <- df[test_index, ]

mean(train_set$churn == "Yes")
mean(test_set$churn == "Yes")
```

For modeling, I went with two approaches, as required by the project instructions:

1. **Logistic regression** - a good baseline for a binary classification problem. It's simple, interpretable, and gives every predictor a coefficient that's easy to talk about.
2. **Random forest** - an ensemble method that can pick up on non-linear relationships and interactions between predictors that logistic regression would miss. I tuned the `mtry` parameter (number of variables tried at each split) over the values 2, 5, 8 and 12, using 200 trees.

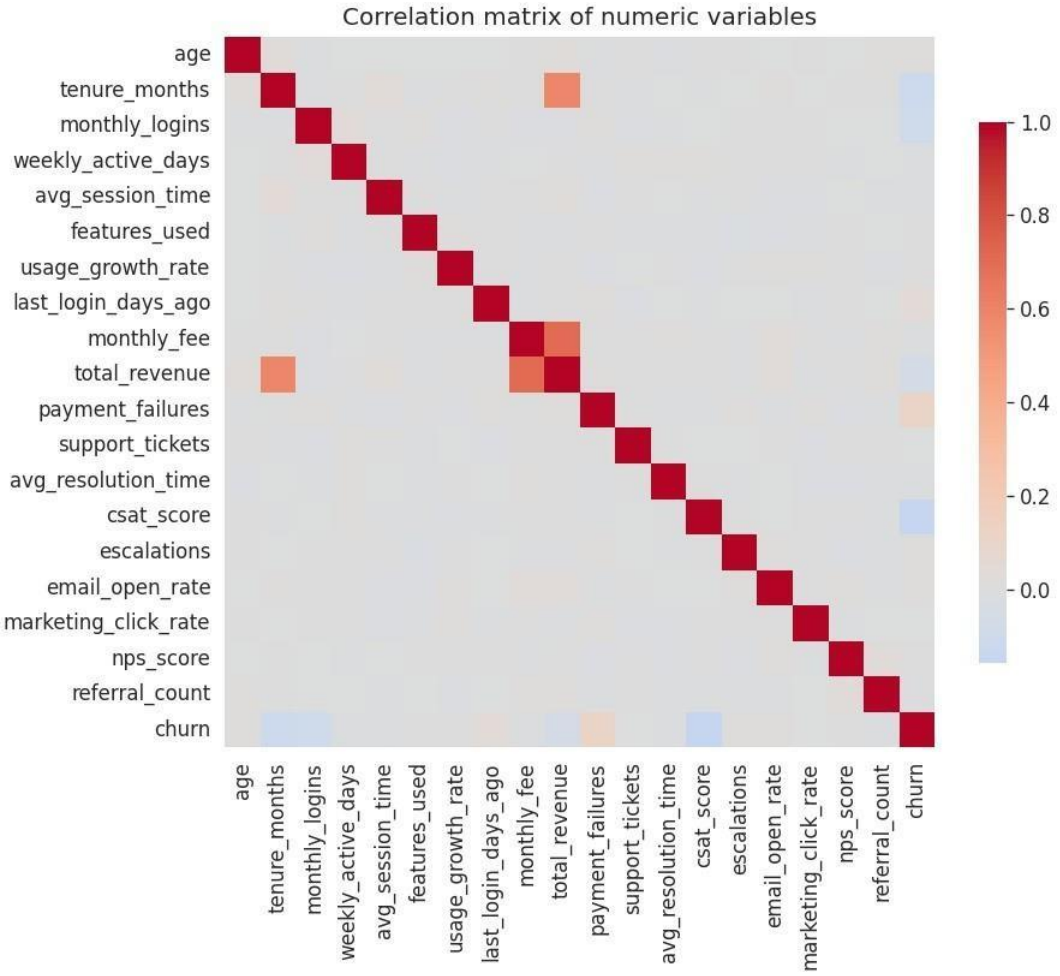

Figure 7: Correlation matrix of numeric variables

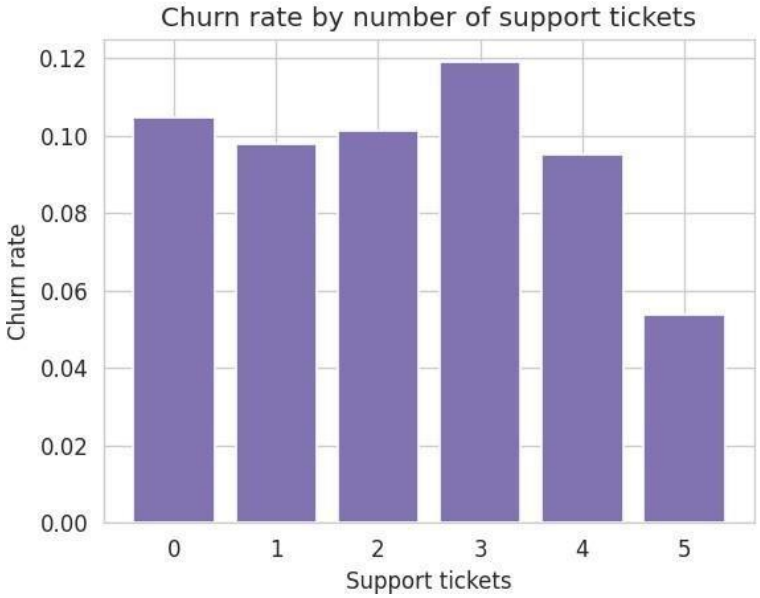

Figure 8: Churn rate by number of support tickets

3 Results

3.1 Logistic regression

```
log_fit <- train(churn ~ ., data = train_set, method = "glm", family = "binomial")
log_pred <- predict(log_fit, test_set)
confusionMatrix(log_pred, test_set$churn, positive = "Yes")
```

The logistic regression model reached an overall **accuracy of 89.6%** on the test set. At first glance that looks pretty good, but the confusion matrix tells a different story:

	Predicted: No	Predicted: Yes
Actual: No	1790	6
Actual: Yes	203	1

The model basically predicted "No" (not churned) for almost every customer. It correctly identified only **1 out of 204** churners in the test set - a sensitivity of about **0.5%**. The specificity (correctly identifying customers who *didn't* churn) was very high, about 99.7%, but that's not very useful if the whole point is to flag at-risk customers. The F1 score, which balances precision and recall, was only about **0.009**.

3.2 Random forest

```
rf_grid <- expand.grid(mtry = c(2, 5, 8, 12))
rf_fit <- train(churn ~ ., data = train_set, method = "rf",
                tuneGrid = rf_grid, ntree = 200, importance = TRUE)
rf_pred <- predict(rf_fit, test_set)
confusionMatrix(rf_pred, test_set$churn, positive = "Yes")
```

The best random forest model (`mtry = 8`) reached an accuracy of **89.9%** on the test set - only marginally better than logistic regression. Its confusion matrix:

	Predicted: No	Predicted: Yes
Actual: No	1794	2
Actual: Yes	200	4

The random forest correctly identified **4 out of 204** churners (sensitivity of about 2.0%), with specificity of about 99.9% and an F1 score of about **0.038**. So the random forest did pick up a few more churners than logistic regression, but the overwhelming majority of churners in the test set were still missed by both models.

3.3 Variable importance

Looking at the random forest's variable importance scores gives some idea of which features the model actually relied on most.

The top features were `tenure_months`, `monthly_logins`, `csat_score`, `total_revenue` and `avg_session_time` - which lines up reasonably well with what I saw during the exploratory analysis (tenure and CSAT both showed visible differences between churners and non-churners).

3.4 Comparing the two models

```
results <- tibble(
  Model = c("Logistic Regression", "Random Forest"),
  Accuracy = c(0.8955, 0.8990),
```

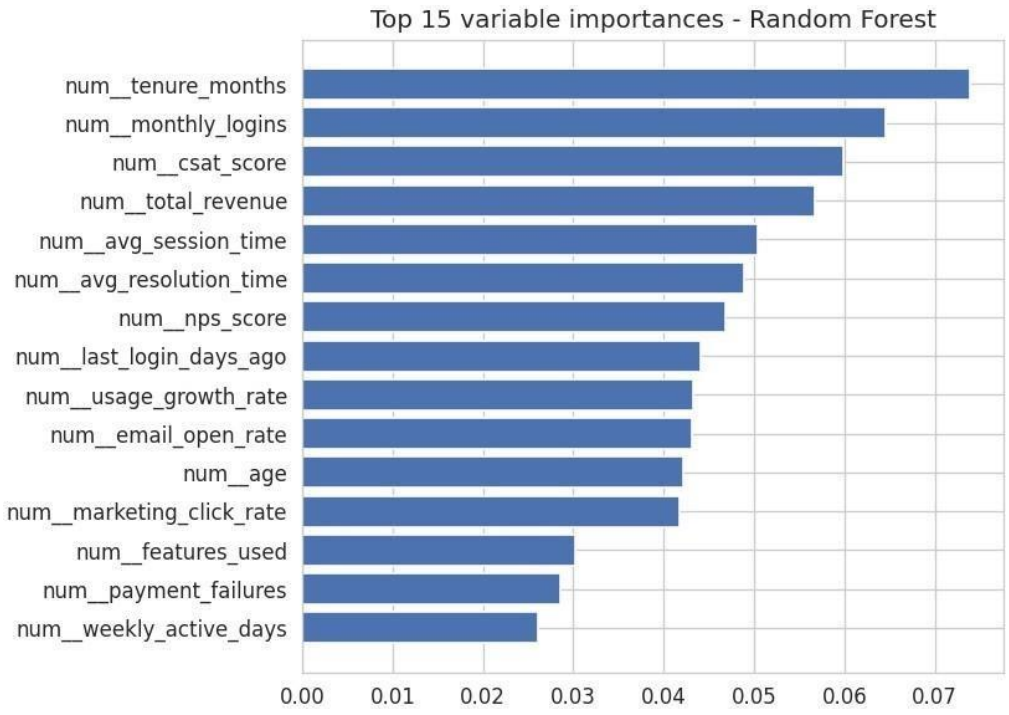

Figure 9: Top 15 variable importances - Random Forest

```
  Sensitivity = c(0.0049, 0.0196),
  Specificity = c(0.9967, 0.9989),
  F1 = c(0.0095, 0.0381)
)
results
```

Model	Accuracy	Sensitivity	Specificity	F1
Logistic Regression	0.8955	0.0049	0.9967	0.0095
Random Forest	0.8990	0.0196	0.9989	0.0381

The random forest beat logistic regression on every metric, but only by a small margin, and both models struggled badly at the actual task of spotting customers who are about to churn.

4 Conclusion

4.1 Summary

In this project I built two classification models (logistic regression and random forest) to predict customer churn using a dataset of 10,000 customers with 30 features covering demographics, usage, billing, support interactions and satisfaction. After cleaning the data (recoding the one column with missing values) and exploring it visually, I found that tenure, CSAT score, and number of payment failures showed the clearest relationships with churn, while variables like contract type, NPS score, and complaint type showed almost no relationship at all.

Both models reached around 89-90% accuracy, but this number is misleading because of how imbalanced the dataset is (only ~10% of customers actually churned). When I looked beyond accuracy at sensitivity, both models missed the large majority of actual churners - logistic regression caught about 0.5% of them and random forest caught about 2%.

4.2 Potential impact

If this kind of model were deployed for a real business, even catching a small extra percentage of at-risk customers could translate into real retention savings, especially if combined with a low-cost intervention (e.g. an automated email offering a discount). However, in its current form, neither model is reliable enough to be the sole basis for a retention campaign - the business would essentially be missing almost all of the customers who are about to leave.

4.3 Limitations

- **Class imbalance**: with only ~10% positive cases, both models default to predicting the majority class most of the time. Accuracy is not a meaningful metric here on its own.
- **Weak individual predictors**: several variables I expected to be useful (contract type, NPS, complaint type) showed almost no difference between churners and non-churners in this dataset, which limits how much signal either model has to work with.
- **Default decision threshold**: both models use the default 0.5 probability cutoff for classifying "Yes"/"No", which, combined with the class imbalance, biases predictions heavily towards "No".

4.4 Future work

- **Address the class imbalance directly**, for example by using techniques like SMOTE/oversampling of the minority class, class weighting, or by tuning the probability threshold to favour recall over raw accuracy.
- **Try other algorithms**, such as gradient boosting (e.g. xgboost) or a regularized logistic regression (e.g. lasso/elastic net) which can sometimes handle imbalanced classification a bit better.
- **Engineer new features**, for example combining `last_login_days_ago` with `monthly_logins` into an "engagement score", which might capture early warning signs of churn better than the raw variables on their own.
- **Use a probability output + threshold tuning** rather than the default classification, and evaluate using a metric like AUC or precision-recall AUC that is more appropriate for imbalanced problems.

5 References

- Dataset: Customer churn business dataset (CSV, 10,000 rows), included with this submission / hosted on GitHub.
- Github: https://github.com/emanahmad95/data_science_capstone/blob/main/customer_churn_business_dataset.csv
- Irizarry, R. A. (2019). *Introduction to Data Science: Data Analysis and Prediction Algorithms with R.*

- Kuhn, M. (2008). Building Predictive Models in R Using the caret Package. *Journal of Statistical Software*, 28(5).
- Liaw, A. & Wiener, M. (2002). Classification and Regression by randomForest. *R News*, 2(3), 18-22.